Verilog 第四次实验报告

实验名称	32 位位计数器与十分频器的设计与仿真				
学生姓名	赵宇哲	学号	2411094	指导老师	杨飞
实验地点	公共实验楼 A 区 306 教室		实验时间	2025-10-22 星期三第三四节	

1 实验项目名称

32 位位计数器与十分频器的设计与仿真

2 实验目的

1. 理解组合逻辑电路与时序逻辑电路的区别
2. 熟悉 `always` 块、`for` 循环、条件语句等的使用方式，使用 Verilog 实现计数器与分频器

3 必修或选修

必修

4 实验平台

Xilinx Vivado Design Suite 2017.1（64-bit）

5 实验内容及步骤

5.1 32 位位计数器

实验内容

设计一个模块，对 32 位输入数据中 1 的个数和 0 的个数进行统计输出。

设计思路

不用一条长链去“累加”，而是分层两两相加，形成一棵平衡二叉加法树，每一层并行计算，逐层把部分和向上汇聚；随着层数上升，加法位宽按需要小幅增加，从而在保持较低关键路径的同时避免不必要的宽加法器。

仿真结果

由波形图可见，`data_in` 依次为 `f0f0f0f0`、`ffffffff`、`00000000`；对应输出中，`count_1`/`count_0` 分别统计 1/0 的个数：当 `data_in=f0f0f0f0` 时两者均为 0x10（各 16 个）；当输入全为 1 时分别为 0x20 与 0x00；当输入全为 0 时分别为 0x00 与 0x20。三组结果均满足 `count_1` + `count_0` = 0x20 (32)，验证模块功能正确、响应及时。

32 位位计数器代码与仿真

5.1.1 设计文件

```
// count01.v
module count01(
    input  [31:0] cin,
    output [5:0]  data0,  // 0 的个数
    output [5:0]  data1   // 1 的个数
);
    // L1: 16 组 2 位和 (0..2)
    wire [1:0] s2 [0:15];
    genvar i;
    generate
        for (i=0; i<16; i=i+1) begin : L1
            assign s2[i] = cin[2*i] + cin[2*i+1];
        end
    endgenerate

    // L2: 8 组 3 位和 (0..4)
    wire [2:0] s4 [0:7];
    generate
        for (i=0; i<8; i=i+1) begin : L2
            assign s4[i] = {1'b0, s2[2*i]} + {1'b0, s2[2*i+1]};
        end
    endgenerate

    // L3: 4 组 4 位和 (0..8)
    wire [3:0] s8 [0:3];
    generate
        for (i=0; i<4; i=i+1) begin : L3
            assign s8[i] = {1'b0, s4[2*i]} + {1'b0, s4[2*i+1]};
        end
    endgenerate

    // L4: 2 组 5 位和 (0..16)
    wire [4:0] s16 [0:1];
    assign s16[0] = {1'b0, s8[0]} + {1'b0, s8[1]};
    assign s16[1] = {1'b0, s8[2]} + {1'b0, s8[3]};

    // L5: 最终 6 位 (0..32)
    wire [5:0] ones = {1'b0, s16[0]} + {1'b0, s16[1]};

    assign data1 = ones;
    assign data0 = 6'd32 - ones;
endmodule
```

5.1.2 仿真文件

```
// tb_count01.v
`timescale 1ns/1ps
module tb_count01;
    reg  [31:0] cin;
    wire [5:0]  data0, data1;

    count01 dut (.cin(cin), .data0(data0), .data1(data1));

    // 期望值计算 + 断言
    task check(input [31:0] v);
        integer i, ones_calc;
        begin
            cin = v; #1;                    // 组合稳定
            ones_calc = 0;
            for (i=0; i<32; i=i+1)
                if (v[i]) ones_calc = ones_calc + 1;

            if (data1 !== ones_calc[5:0] || data0 + data1 !== 6'd32) begin
                $display("ERROR cin=%h  expect ones=%0d got=%0d (zeros=%0d)",
                         v, ones_calc, data1, data0);
                $stop;
            end else begin
                $display("OK    cin=%h  zeros=%0d ones=%0d", v, data0, data1);
            end
        end
    endtask

    integer t; // 顶层循环变量（仅仿真用）
    initial begin
        // 可选：生成波形
        $dumpfile("count01.vcd");
        $dumpvars(0, tb_count01);

        // 基本用例
        check(32'h0000_0000);
        check(32'hFFFF_FFFF);
        check(32'h0123_4567);
        check(32'h89AB_CDEF);

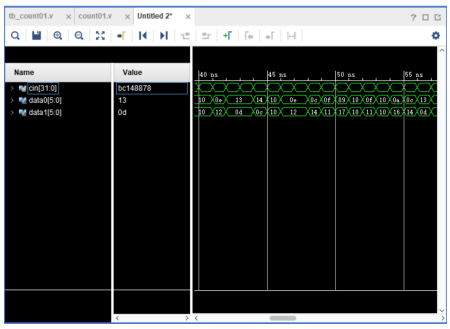
        // 单比特滑动（不使用额外信号）
        for (t=0; t<32; t=t+1)
            check(32'h1 << t);

        // 随机若干次
        repeat (100) check($random);

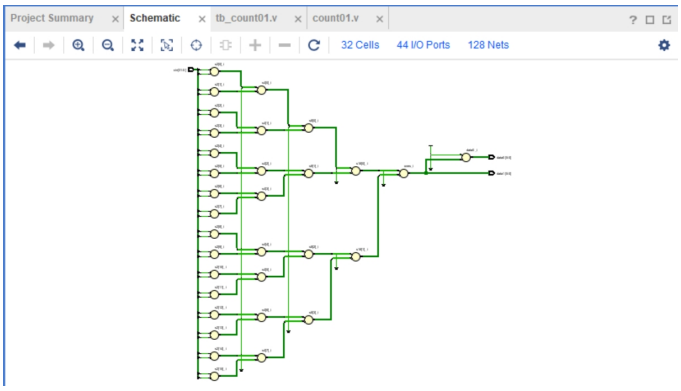
        $display("All tests passed.");
        $finish;
    end
endmodule
```

32 位位计数器代码与仿真

5.1.3 仿真结果



32 位位计数器波形图



32 位位计数器电路

5.2 十分频器

实验内容

设计一个模块，对 32 位输入数据中 1 的个数和 0 的个数进行统计输出。

设计思路

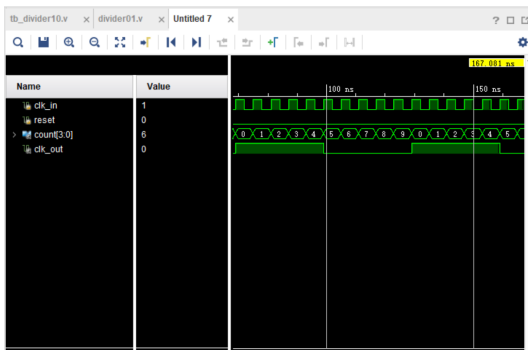
不用一条长链去“累加”，而是分层两两相加，形成一棵平衡二叉加法树，每一层并行计算，逐层把部分和向上汇聚；随着层数上升，加法位宽按需要小幅增加，从而在保持较低关键路径的同时避免不必要的宽加法器。

仿真结果

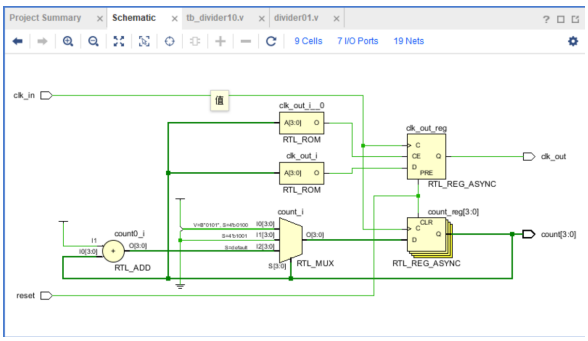
由波形图可见，`data_in` 依次为 `f0f0f0f0`、`ffffffff`、`00000000`；对应输出中，`count_1/count_0` 分别统计 1/0 的个数：当 `data_in=f0f0f0f0` 时两者均为 0x10（各 16 个）；当输入全为 1 时分别为 0x20 与 0x00；当输入全为 0 时分别为 0x00 与 0x20。三组结果均满足 `count_1` + `count_0` = 0x20 (32)，验证模块功能正确、响应及时。

十分频器代码与仿真

5.2.1 仿真结果



十分频器波形图



十分频器电路

十分频器代码与仿真

5.2.2 设计文件

```v
// devider10.v
module divider10 (
    input  wire       clk_in,   // 输入时钟
    input  wire       reset,    // 异步复位，高有效
    output reg  [3:0] count,    // 0~9 计数
    output reg        clk_out   // 输出时钟（÷10）
);

    // 异步复位 + 同步计数
    always @(posedge clk_in or posedge reset) begin
        if (reset) begin
            count   <= 4'd0;    // 复位后从 0 开始
            clk_out <= 1'b1;    // 要求：复位后先为高电平
        end else begin
            case (count)
                4'd4: begin
                    // 下一拍进入 5，同时把输出拉低（高电平持续 5 拍：0~4）
                    count   <= 4'd5;
                    clk_out <= 1'b0;
                end
                4'd9: begin
                    // 计满 9 回到 0，同时把输出拉高（低电平持续 5 拍：5~9）
                    count   <= 4'd0;
                    clk_out <= 1'b1;
                end
                default: begin
                    count   <= count + 4'd1;
                    clk_out <= clk_out; // 其余拍保持不变
                end
            endcase
        end
    end

endmodule
```

5.2.3 仿真文件

```v
// tb_devider10.v
`timescale 1ns/1ps

module tb_divider10;

    reg        clk_in;
    reg        reset;
    wire [3:0] count;
    wire       clk_out;

    // DUT
    divider10 dut (
        .clk_in (clk_in),
        .reset  (reset),
        .count  (count),
        .clk_out(clk_out)
    );

    // 时钟：6ns 周期（~166.7 MHz）
    initial begin
        clk_in = 1'b0;
        forever #3 clk_in = ~clk_in;
    end

    // 复位与结束
    initial begin
        reset = 1'b1;
        #10;
        reset = 1'b0;
        #400;       // 观察若干个输出周期
        $finish;
    end

    // 波形转储
    initial begin
        $dumpfile("divider10.vcd");
        $dumpvars(0, tb_divider10);
    end

    // 便于观察的打印
    initial $display(" time  | count clk_out");
    always @(posedge clk_in) $display("%5t |   %1d     %b"
      , $time, count, clk_out);

endmodule
```

6 关键问题讨论

- 计数器优化：32 位计数器没有用简单的长链累加，而是用了“平衡二叉加法树”的结构。这种分层、并行的设计（如代码中的 s2, s4, s8...）能缩短关键路径，提高电路速度。
- 异步复位：十分频器使用了异步复位。reset 信号一旦有效（高电平），会立即将计数器清零 4'd0 并将输出置为 1'b1，不受时钟信号 clk_in 的影响。

7 总结

在 32 位计数器中，我实践了组合逻辑的优化，采用加法树结构替代长链累加，仿真结果验证了其功能正确。在十分频器中，我掌握了时序逻辑的设计，通过 0-9 计数器和 case 语句实现了 50% 占空比的分频。